

Week 5 Lab Worksheet

The objective of this Week 5 laboratory session was to explore the architectural paradigm of Software-Defined Networking (SDN) by decoupling the control plane from the data plane. By deploying the ONOS controller and integrating it with a Mininet emulation, we evaluated how modular applications dictate network behaviour (LO1 & LO2). Furthermore, by utilizing Wireshark to analyse Address Resolution Protocol (ARP) traffic, we empirically demonstrated how SDN applications like proxyarp can optimize network performance by suppressing data-plane broadcast storms (LO3). Finally, we utilized the ONOS GUI to visualize real-time network topology and telemetry (LO4)

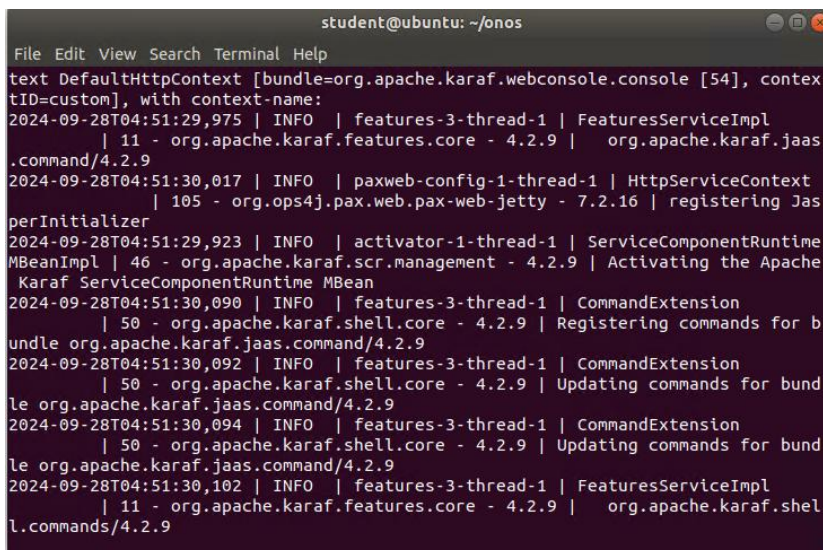
Learning Objectives

1. Deploy, build, and orchestrate a distributed Software-Defined Networking (SDN) control plane.
2. Dynamically provision network topologies and evaluate the efficacy of modular SDN applications
3. Empirically analyse the impact of control-plane logic on data-plane broadcast traffic.
4. Visualize and audit real-time network state, topology, and telemetry via SDN Graphical User Interfaces.

1. Software Defined Networking (SDN)

1.Run ONOS Controller

- a. Open a Terminal and Type
 - i. `cd $ONOS_ROOT`
 - ii. `bazel build onos`
 - iii. `runsdn`



```

student@ubuntu: ~/onos
File Edit View Search Terminal Help
text DefaultHttpContext [bundle=org.apache.karaf.webconsole.console [54], contextID=custom], with context-name:
2024-09-28T04:51:29,975 | INFO | features-3-thread-1 | FeaturesServiceImpl
| 11 - org.apache.karaf.features.core - 4.2.9 | org.apache.karaf.jaas
.command/4.2.9
2024-09-28T04:51:30,017 | INFO | paxweb-config-1-thread-1 | HttpServiceContext
| 105 - org.ops4j.pax.web.pax-web-jetty - 7.2.16 | registering JasperInitializer
2024-09-28T04:51:29,923 | INFO | activator-1-thread-1 | ServiceComponentRuntime
MBeanImpl | 46 - org.apache.karaf.scr.management - 4.2.9 | Activating the Apache
Karaf ServiceComponentRuntime MBean
2024-09-28T04:51:30,090 | INFO | features-3-thread-1 | CommandExtension
| 50 - org.apache.karaf.shell.core - 4.2.9 | Registering commands for bundle
org.apache.karaf.jaas.command/4.2.9
2024-09-28T04:51:30,092 | INFO | features-3-thread-1 | CommandExtension
| 50 - org.apache.karaf.shell.core - 4.2.9 | Updating commands for bundle
org.apache.karaf.jaas.command/4.2.9
2024-09-28T04:51:30,094 | INFO | features-3-thread-1 | CommandExtension
| 50 - org.apache.karaf.shell.core - 4.2.9 | Updating commands for bundle
org.apache.karaf.jaas.command/4.2.9
2024-09-28T04:51:30,102 | INFO | features-3-thread-1 | FeaturesServiceImpl
| 11 - org.apache.karaf.features.core - 4.2.9 | org.apache.karaf.shell
l.commands/4.2.9

```

This will load the required libraries and start the ONOS environment.

In another terminal, push the configurations to Mininet by running your Python script

2. Navigate to the folder 7COM2008/LAB4 and Right Click and Open in Terminal
 - a. `sudo python3 lab4.py`
In mininet terminal
 - i. `pingall`
This ping will fail.
 - b. Open another terminal and type
 - i. `onos localhost`

```

student@ubuntu: ~
File Edit View Search Terminal Help
student@ubuntu:~$ onos localhost
Welcome to Open Network Operating System (ONOS)!

  ONOS

Documentation: wiki.onosproject.org
Tutorials:    tutorials.onosproject.org
Mailing lists: lists.onosproject.org

Come help out! Find out how at: contribute.onosproject.org

Hit '<tab>' for a list of available commands
and '[cmd] --help' for help on a specific command.
Hit '<ctrl-d>' or type 'logout' to exit ONOS session.

student@root >
04:54:21

```

2. Applications and SDN

1. In the onos localhost terminal, type in the following
 - a. `app activate org.onosproject.fwd`
 - b. `app activate org.onosproject.openflow`

`fwd` application activates the Reactive Forwarding application. The `fwd` application acts as a foundational, "plug-and-play" tool that enables basic network connectivity. While the `fwd` (Reactive Forwarding) application handles the high-level logic of *how* traffic moves, the OpenFlow application is the underlying engine and translator that allows ONOS to physically talk to network switches

2. On mininet terminal, now try
 - a. `pingall`
 - b. `exit mininet`
3. Open wireshark and capture on s1-eth1 (use another terminal)
4. Re-run the topology (make sure you are in the correct folder (7COM2008/LAB4))
 - a. `sudo python3 lab4.py`
 - b. Whilst observing s1-eth1, ping h1 and h4
 - i. `h1 ping h4`

You will see ARP packets like this below

31	17.308212111	00:00:00 00:00:01	Broadcast	ARP	42 Who has 10.0.0.4? tell 10.0.0.1
32	17.310752866	00:00:00 00:00:04	00:00:00 00:00:01	ARP	42 10.0.0.4 is at 00:00:00:00:00:04

- c. `exit mininet`
5. Launch the mininet topology again
 - a. `sudo python3 lab4.py`
 - b. In the onos localhost terminal, type in the following
 - i. `app activate org.onosproject.proxyarp`

`proxyarp` Allows the ONOS controller to answer ARP discovery queries locally on behalf of hosts rather than broadcasting them globally. This drastically cuts down background broadcast storms

- c. Now ping h1 and h4
 - i. `h1 ping h4`

Now you will not see the ARP messages as the controller answered and resolved them.
6. You can also observe global network using the GUI at:

<http://localhost:8181/onos/ui>

Copy the above link to Firefox

7. Keep a ping between two hosts (`h2 ping h4`) and try the following hot keys in the ONOS GUI
 - a. A
 - b. L
 - c. H